

Thiết kế bộ SC FIFO, DC FIFO

1. Mục tiêu thiết kế

Mục tiêu của bài thực hành này là thiết kế, mô phỏng và đánh giá hai loại bộ đệm dữ liệu (Buffer) phổ biến nhất trong vi mạch:

- **SC FIFO (Synchronous FIFO):** Hệ thống đồng bộ, dùng chung một nguồn xung nhịp (Clock) cho cả quá trình đọc và ghi.
- **DC FIFO (Asynchronous/Dual-Clock FIFO):** Hệ thống bất đồng bộ, sử dụng hai nguồn xung nhịp độc lập cho quá trình đọc và ghi, giải quyết bài toán giao tiếp xuyên miền xung nhịp (Clock Domain Crossing - CDC).

A. THIẾT KẾ SC FIFO (SYNCHRONOUS FIFO)

1. Tổng quan

SC FIFO (Synchronous First-In, First-Out) là bộ đệm dữ liệu đồng bộ, hoạt động theo nguyên tắc "Vào trước - Ra trước". Từ "Đồng bộ" (Synchronous) có nghĩa là toàn bộ hệ thống, bao gồm cả thao tác Đọc (Read) và thao tác Ghi (Write), đều được điều khiển bởi cùng một nguồn xung nhịp (Clock) duy nhất.

Ứng dụng cốt lõi của SC FIFO trong thiết kế vi mạch là làm bộ đệm (Buffer) để dung hòa tốc độ truyền nhận dữ liệu giữa hai module (Producer và Consumer) hoạt động chung một miền xung nhịp, giúp hệ thống không bị nghẽn khi có luồng dữ liệu lớn đổ về đột ngột (Bursty data).

2. Kiến trúc và nguyên lý hoạt động

Thiết kế SC FIFO trong bài thực hành này được xây dựng dựa trên kiến trúc **Dual-Port RAM** (RAM cổng kép) kết hợp với bộ đệm vòng (Circular Buffer), cho phép vừa đọc vừa ghi trong cùng một chu kỳ Clock. Các thành phần chính bao gồm:

- **Bộ nhớ (Memory Array):** Mảng thanh ghi (reg array) lưu trữ dữ liệu.
- **Con trỏ Ghi (wr_ptr) và Con trỏ Đọc (rd_ptr):** Tự động tăng lên 1 sau mỗi lần ghi/đọc thành công. Đặc biệt, các con trỏ này được thiết kế **rộng hơn địa chỉ thực tế 1 bit (N+1 bits)**. Bit cao nhất (MSB) đóng vai trò làm cờ đánh dấu số vòng quay của con trỏ, các bit còn lại dùng để trỏ vào địa chỉ RAM.
- **Logic tạo cờ trạng thái (Status Flags):**
 - o empty (Cờ Rỗng): Bật lên 1 khi wr_ptr và rd_ptr giống nhau hoàn toàn (chưa có dữ liệu hoặc đã đọc hết). Hệ thống sẽ chặn lệnh đọc (re) khi cờ này bật để chống lỗi Underflow.
 - o full (Cờ Đầy): Bật lên 1 khi bit MSB của 2 con trỏ khác nhau (con trỏ ghi đã chạy nhanh hơn đúng 1 vòng), nhưng các bit địa chỉ bên dưới thì giống hệt nhau. Hệ thống sẽ chặn lệnh ghi (we) khi cờ này bật để chống lỗi Overflow.

3. Code verilog thiết kế SC FIFO

```
module sc_fifo #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 4
)(
    input clk, rst,
    input re,
    input we,
    input [DATA_WIDTH-1:0] din,
    output reg [DATA_WIDTH-1:0] dout,
    output wire full, empty
);

    localparam DEPTH = (1 << ADDR_WIDTH);

    (*ram_style = "block" *)
    reg [DATA_WIDTH-1:0] mem[0:DEPTH-1];
    reg [ADDR_WIDTH:0] wr_ptr, rd_ptr;

    assign empty = (wr_ptr == rd_ptr);
    assign full = ((wr_ptr[ADDR_WIDTH] != rd_ptr[ADDR_WIDTH]) &&
    (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr[ADDR_WIDTH-1:0]));

    always @(posedge clk) begin
        if (rst)
            wr_ptr <= 0;
        else begin
            if (we && !full) begin
                mem[wr_ptr[ADDR_WIDTH-1:0]] <= din;
                wr_ptr <= wr_ptr + 1;
            end
        end
    end

    always @(posedge clk) begin
        if (rst) begin
            rd_ptr <= 0;
            dout <= 0;
        end else begin
            if (re && !empty) begin
                dout <= mem[rd_ptr[ADDR_WIDTH-1:0]];
                rd_ptr <= rd_ptr + 1;
            end
        end
    end
endmodule
```

4. Testbench

```
module tb_sc_fifo;

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 4;

    reg clk;
    reg rst;
    reg we;
    reg re;
    reg [DATA_WIDTH-1:0] din;

    wire [DATA_WIDTH-1:0] dout;
    wire full;
    wire empty;

    sc_fifo #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) dut (
        .clk(clk),
        .rst(rst),
        .re(re),
        .we(we),
        .din(din),
        .dout(dout),
        .full(full),
        .empty(empty)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    integer i;

    initial begin
        // Khởi tạo
        rst = 1;
        we = 0;
        re = 0;
        din = 0;
```

```

#25;
@(negedge clk);
rst = 0;

#10;

$display("[%0t] BAT DAU GHI 16 GIA TRI VAO FIFO...", $time);
for (i = 0; i < 16; i = i + 1) begin
    @(negedge clk);
    we = 1;
    din = i + 8'hA0; // Lan luot ghi A0, A1, A2... den AF
end

@(negedge clk);
$display("[%0t] FIFO DA DAY! Co full = %b", $time, full);
we = 1;
din = 8'hFF;

@(negedge clk);
we = 0; // Tat lenh ghi

#20;

$display("[%0t] BAT DAU RUT DU LIEU RA...", $time);
for (i = 0; i < 16; i = i + 1) begin
    @(negedge clk);
    re = 1;
end

@(negedge clk);
$display("[%0t] FIFO DA RONG! Co empty = %b", $time, empty);
re = 1;

@(negedge clk);
re = 0; // Tat lenh doc

#20;

$display("[%0t] BAT DAU TEST VUA DOC VUA GHI CUNG LUC...", $time);

// Ghi moi truoc 2 gia tri vao FIFO
@(negedge clk); we = 1; re = 0; din = 8'h11;
@(negedge clk); we = 1; re = 0; din = 8'h22;

// Kich hoat CA DOC VA GHI trong cung mot chu ky clock

```

```

@(negedge clk);
we = 1; din = 8'h33; // Ghi vao 33
re = 1;           // Dong thoi lay du lieu cu ra

@(negedge clk);
we = 1; din = 8'h44; // Ghi vao 44
re = 1;           // Lay du lieu ra

@(negedge clk);
we = 0;
re = 0;

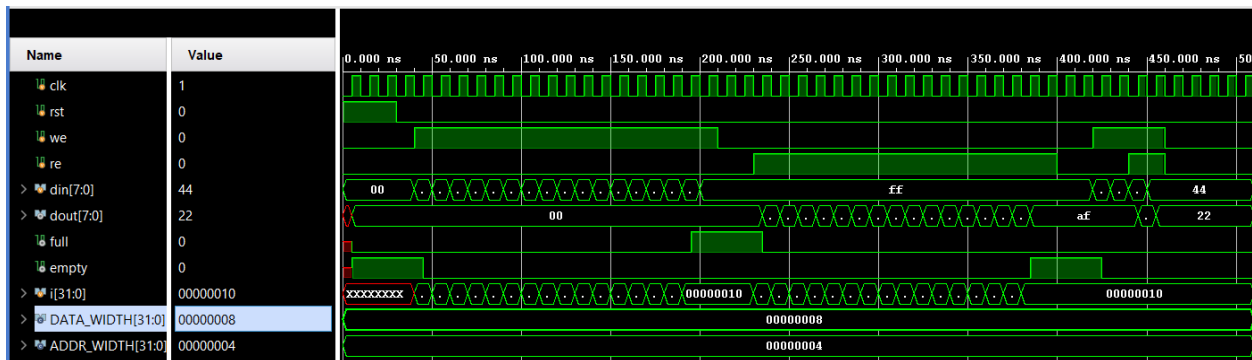
#50;
$display("[%0t] HOAN TAT MO PHONG!", $time);
$finish;
end

endmodule

```

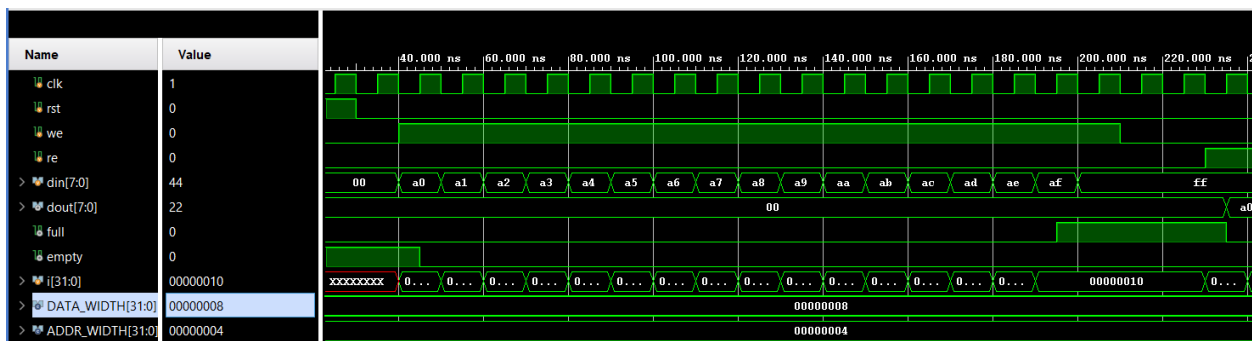
5. Kết quả mô phỏng

5.1. Mô phỏng Waveform



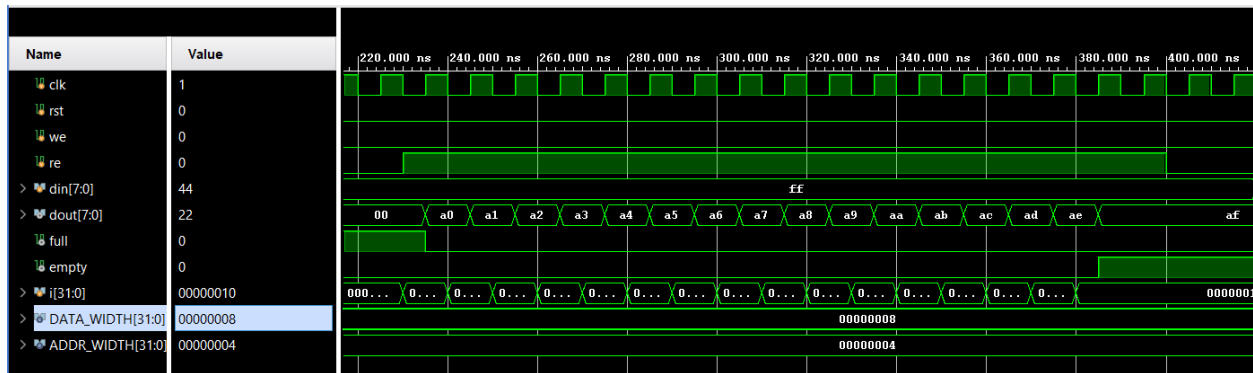
Hình 1 Waveform SC FIFO

- 45 – 195ns: Thực hiện ghi đến khi FIFO đầy



Hình 2 phần Waveform cụ thể của chức năng ghi

- 235 – 385ns: Thực hiện đọc đến khi FIFO rỗng (theo thứ tự các giá trị đã ghi vào → FIFO)



Hình 3 phần Waveform cụ thể của chức năng đọc

- 200 – 235ns: tiếp tục ghi ff khi FIFO đã đầy
 - o 385ns: sau khi đọc giá trị af ra, FIFO đã báo rỗng → giá trị ff không được ghi
- 425 – 435ns: we = 1 → ghi 11 và 22 vào FIFO
- 445ns: we = 1 & re = 1: ghi 33 vào và đọc 11 → dual port

5.2. Kết quả Console

```
[40000] BAT DAU GHI 16 GIA TRI VAO FIFO...
[200000] FIFO DA DAY! Co full = 1
[230000] BAT DAU RUT DU LIEU RA...
[390000] FIFO DA RONG! Co empty = 1
[420000] BAT DAU TEST VUA DOC VUA GHI CUNG LUC...
[510000] HOAN TAT MO PHONG!
```

Hình 4 Kết quả console SC FIFO

- Thông báo cụ thể tại từng thời gian thực hiện chức năng cố định

B. THIẾT KẾ DC FIFO (ASYNCHRONOUS FIFO)

1. Tổng quan và Thách thức (Clock Domain Crossing)

DC FIFO phức tạp hơn SC FIFO rất nhiều do dữ liệu được ghi bởi wr_clk và đọc bởi rd_clk có tần số hoàn toàn lệch pha nhau. Điều này sinh ra hai thách thức cực kỳ nghiêm trọng:

- **Trạng thái bán chuẩn (Metastability):** Khi một Flip-flop ở miền xung nhịp này cố gắng "chụp" tín hiệu đang chuyển trạng thái từ miền xung nhịp khác, ngõ ra sẽ rơi vào trạng thái lơ lửng, gây treo mạch.
- **Lỗi lấy mẫu con trỏ (Pointer Sampling Error):** Khi nhiều bit của con trỏ nhị phân thay đổi cùng lúc (ví dụ từ 0111 lên 1000), miền nhận có thể bắt sai giá trị do độ trễ đường dây khác nhau.

2. Kiến trúc cốt lõi của DC FIFO

Để giải quyết bài toán CDC, DC FIFO tích hợp 2 module phần cứng đặc biệt:

- **Bộ mã hóa Gray (Gray Code):** Con trỏ nhị phân được chuyển sang mã Gray trước khi gửi sang miền xung nhịp khác. Đặc điểm của mã Gray là khi tăng/giảm 1 đơn vị, chỉ có **duy nhất 1 bit thay đổi trạng thái**, loại bỏ hoàn toàn rủi ro đọc sai mớ bit đang nhảy loạn xạ.
- **Bộ đồng bộ 2 tầng (2-Stage Synchronizer):** Dùng 2 Flip-flop mắc nối tiếp nhau tại miền nhận để triệt tiêu hoàn toàn trạng thái Metastability.

3. Code verilog thiết kế DC FIFO

3.1. Module mã hóa Gray

```
module bin_to_gray #(
    parameter WIDTH = 4
)(
    input  wire [WIDTH-1:0] bin,
    output wire [WIDTH-1:0] gray
);

    assign gray = bin ^ (bin >> 1);

endmodule
```

3.2. Module đồng bộ 2 tầng

```
module synchronizer #(
    parameter WIDTH = 4
)(
    input wire      clk,
    input wire      rst,
    input wire [WIDTH-1:0] d_in,
    output reg [WIDTH-1:0] d_out
);

    reg [WIDTH-1:0] q1;

    always @(posedge clk) begin
        if (rst) begin
            q1  <= 0;
            d_out <= 0;
        end else begin
            q1  <= d_in;
            d_out <= q1;
        end
    end

endmodule
```


3.3. Module DC FIFO

```

module dc_fifo #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 4
)(
    // Giao tiep mien Write (wr_clk)
    input wire        wr_clk,
    input wire        wr_rst,
    input wire        we,
    input wire [DATA_WIDTH-1:0] din,
    output wire        full,

    // Giao tiep mien Read (rd_clk)
    input wire        rd_clk,
    input wire        rd_rst,
    input wire        re,
    output reg [DATA_WIDTH-1:0] dout,
    output wire        empty
);

    localparam DEPTH = (1 << ADDR_WIDTH);

    // Bo nho RAM
    (* ram_style = "block" *)
    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    // Con tro Binary
    reg [ADDR_WIDTH:0] wr_ptr_bin;
    reg [ADDR_WIDTH:0] rd_ptr_bin;

    // Con tro Gray
    wire [ADDR_WIDTH:0] wr_ptr_gray;
    wire [ADDR_WIDTH:0] rd_ptr_gray;

    // Con tro Gray da duoc dong bo
    wire [ADDR_WIDTH:0] wr_ptr_gray_sync;
    wire [ADDR_WIDTH:0] rd_ptr_gray_sync;

    // Mien Write (wr_clk)
    always @(posedge wr_clk) begin
        if (wr_rst) begin
            wr_ptr_bin <= 0;
        end else if (we && !full) begin
            mem[wr_ptr_bin[ADDR_WIDTH-1:0]] <= din;
            wr_ptr_bin <= wr_ptr_bin + 1;
        end
    end
end

```

```

bin_to_gray #(.WIDTH(ADDR_WIDTH + 1)) b2g_wr (
    .bin(wr_ptr_bin),
    .gray(wr_ptr_gray)
);

synchronizer #(.WIDTH(ADDR_WIDTH + 1)) sync_rd2wr (
    .clk(wr_clk),
    .rst(wr_rst),
    .d_in(rd_ptr_gray),
    .d_out(rd_ptr_gray_sync)
);

// Tính toán có Full (So sánh mã Gray: 2 bit cao nhất đảo ngược)
assign full = (wr_ptr_gray == {~rd_ptr_gray_sync[ADDR_WIDTH:ADDR_WIDTH-1],
    rd_ptr_gray_sync[ADDR_WIDTH-2:0]});

// Mien Read (rd_clk)
always @(posedge rd_clk) begin
    if (rd_rst) begin
        rd_ptr_bin <= 0;
        dout <= 0;
    end else if (re && !empty) begin
        dout <= mem[rd_ptr_bin[ADDR_WIDTH-1:0]];
        rd_ptr_bin <= rd_ptr_bin + 1;
    end
end

bin_to_gray #(.WIDTH(ADDR_WIDTH + 1)) b2g_rd (
    .bin(rd_ptr_bin),
    .gray(rd_ptr_gray)
);

synchronizer #(.WIDTH(ADDR_WIDTH + 1)) sync_wr2rd (
    .clk(rd_clk),
    .rst(rd_rst),
    .d_in(wr_ptr_gray),
    .d_out(wr_ptr_gray_sync)
);

// Tính toán có Empty (So sánh mã Gray: giống nhau hoàn toàn)
assign empty = (rd_ptr_gray == wr_ptr_gray_sync);

endmodule

```

4. Testbench

```
`timescale 1ns / 1ps

module tb_dc_fifo;

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 4;

    // Tín hiệu miền Write (100MHz)
    reg wr_clk;
    reg wr_rst;
    reg we;
    reg [DATA_WIDTH-1:0] din;
    wire full;

    // Tín hiệu miền Read (40MHz)
    reg rd_clk;
    reg rd_rst;
    reg re;
    wire [DATA_WIDTH-1:0] dout;
    wire empty;

    dc_fifo #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) dut (
        .wr_clk(wr_clk),
        .wr_rst(wr_rst),
        .we(we),
        .din(din),
        .full(full),
        .rd_clk(rd_clk),
        .rd_rst(rd_rst),
        .re(re),
        .dout(dout),
        .empty(empty)
    );

    // Tạo xung nhịp bất đồng bộ
    initial begin
        wr_clk = 0;
        forever #5 wr_clk = ~wr_clk; // Chu kỳ 10ns (100MHz)
    end

    initial begin
        rd_clk = 0;
    end
```

```

    forever #12.5 rd_clk = ~rd_clk; // Chu kỳ 25ns (40MHz)
end

integer i;

initial begin
    // KHỞI TẠO VÀ RESET HỆ THỐNG
    wr_rst = 1; rd_rst = 1;
    we = 0; re = 0; din = 0;

    #50;
    @(negedge wr_clk) wr_rst = 0;
    @(negedge rd_clk) rd_rst = 0;

    #50;

    // BƠM ĐẦY TỐC ĐỘ CAO & CỐ TÌNH GHI TRÀN (OVERFLOW)
    $display("[%0t] CASE 1: Ghi liên tục 16 giá trị (100MHz)", $time);
    for (i = 0; i < 16; i = i + 1) begin
        @(negedge wr_clk);
        we = 1;
        din = i + 8'hA0;
    end

    // Ghi thêm khi đã đầy (kiểm tra chống tràn)
    @(negedge wr_clk);
    we = 1; din = 8'hFF; // Giá trị FF không được phép lọt vào RAM
    $display("[%0t] Co full = %b. Đang có ghi đè FF vào RAM...", $time, full);

    @(negedge wr_clk);
    we = 0;

    #100;

    // RÚT CẠN TỐC ĐỘ CHẬM & CỐ TÌNH ĐỌC LỖ (UNDERFLOW)
    $display("[%0t] CASE 2: Đọc liên tục ra khỏi FIFO (40MHz)", $time);
    for (i = 0; i < 16; i = i + 1) begin
        @(negedge rd_clk);
        re = 1;
    end

    // Đọc thêm khi đã rỗng (kiểm tra chống underflow)
    @(negedge rd_clk);
    re = 1;
    $display("[%0t] Co empty = %b. Đang có đọc khi FIFO rỗng...", $time, empty);

```

```

@(negedge rd_clk);
re = 0;

#100;

// GHI VÀ ĐỌC ĐỒNG THỜI XUYÊN MIỀN CLOCK (CDC TEST)
$display("[%0t] CASE 3: Vua doc vua ghi voi 2 Clock lech pha", $time);

for (i = 0; i < 3; i = i + 1) begin
    @(negedge wr_clk);
    we = 1; din = 8'hB0 + i;
end
@(negedge wr_clk) we = 0;

#20;

// Bật cả Read và Write đồng thời.
// Do wr_clk (100MHz) nhanh hơn rd_clk (40MHz), FIFO sẽ dần dần bị đầy.
we = 1;
re = 1;

for (i = 0; i < 20; i = i + 1) begin
    @(negedge wr_clk);
    din = din + 1; // Dữ liệu ghi thay đổi liên tục
end

we = 0;
re = 0;

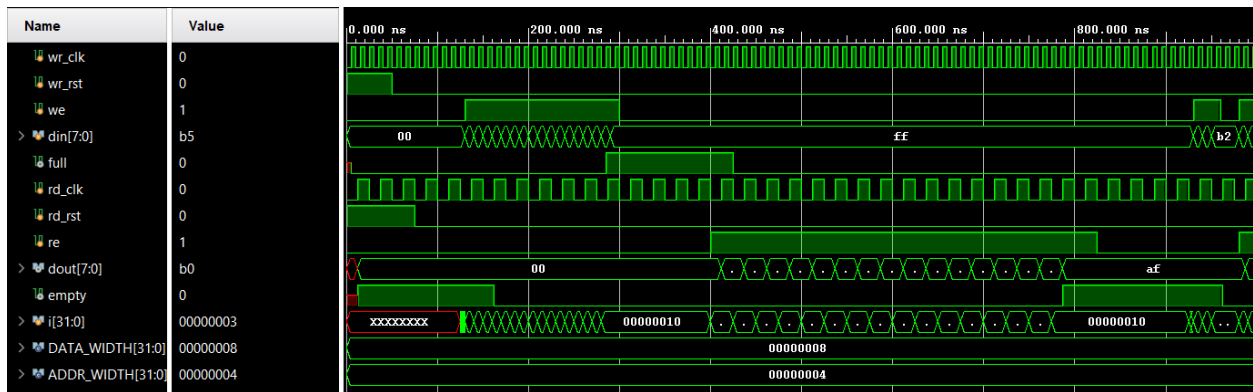
#200;
$display("[%0t] HOAN TAT MO PHONG!", $time);
$finish;
end

endmodule

```

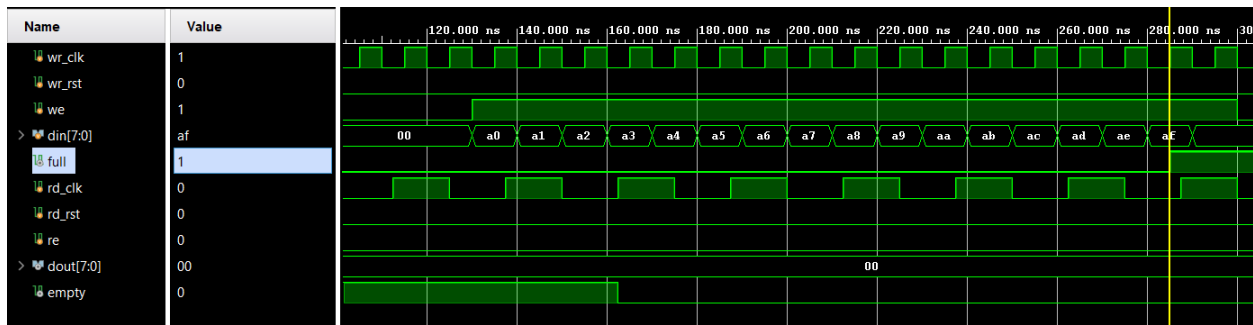
5. Kết quả mô phỏng

5.1. Mô phỏng Waveform



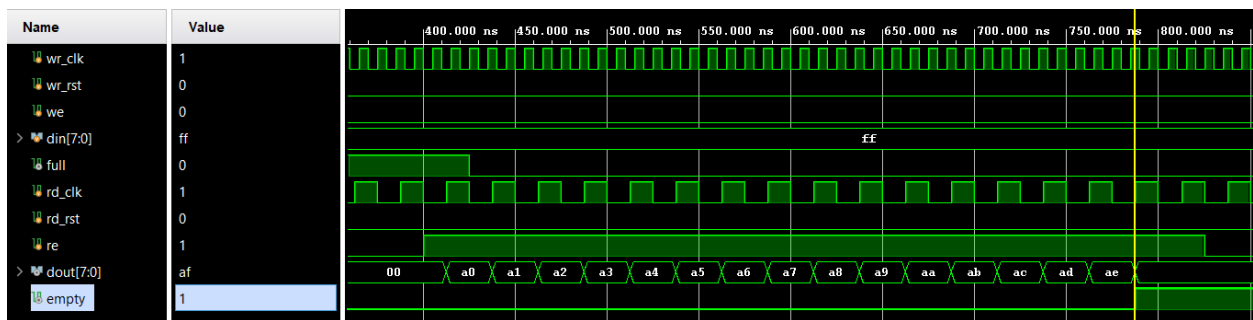
Hình 5 Waveform DC FIFO

- 135 – 285ns: Ghi cho đến khi đầy, giá trị cập nhật theo wr_clk



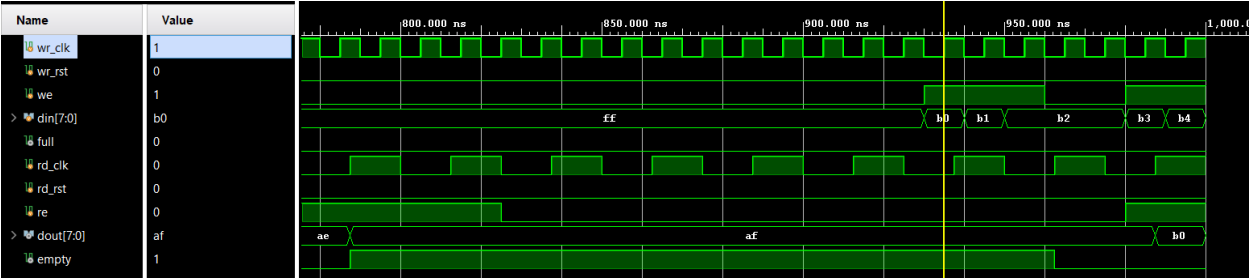
Hình 6 phần Waveform cụ thể cho chức năng ghi

- 412.5 – 787.5ns: Đọc cho đến khi trống, giá trị cập nhật theo rd_clk



Hình 7 phần Waveform cụ thể cho chức năng đọc

- Sau 285ns, thực hiện ghi ff khi FIFO đã báo full → Có thể thấy khi đọc, chỉ đọc tới af và báo empty → ff không được ghi vào
- Sau 787.5ns, thực hiện đọc khi FIFO đã báo empty → dout vẫn giữ nguyên giá trị và empty vẫn báo
- 935ns: ghi b0 vào FIFO → 987.5ns: b0 mới được đọc



Hình 8 phần Waveform cụ thể cho chức năng ghi đọc song song

5.2. Kết quả Console

```
[125000] CASE 1: Ghi lien tục 16 gia tri (100MHz)
[290000] Co full = 1. Dang co ghi de FF vao RAM...
[400000] CASE 2: Doc lien tục ra khoi FIFO (40MHz)
[800000] Co empty = 1. Dang co doc khi FIFO rong...
[925000] CASE 3: Vua doc vua ghi voi 2 Clock lech pha
```

Hình 9 Kết quả trong màn hình Console

- Thông báo thời gian thực hiện chức năng cố định